



Rekursi dan Struct

Base Case • Recursive Call • Tipe Data Bentukkan

Algoritma Pemrograman • IF25-11001 • 2 SKS Teori

Institut Teknologi Sumatera — Teknik Informatika

Review Pertemuan 13

Fungsi dan Modularitas — pondasi untuk rekursi dan struct

Fungsi = Modularitas + DRY

Pecah program besar jadi modul. Parameter (input), return (output), scope (lokal vs global).

Pass by Value vs Reference

Value = salinan (aman). Reference = asli (mengubah). Swap harus pakai reference.

Hari ini: 2 topik baru!

REKURSI: fungsi memanggil diri sendiri. STRUCT: tipe data gabungan (nama+NIM+IPK).

Capaian Pembelajaran Pertemuan 14

Sub-CPMK 14.1

C2 – Menjelaskan

Menjelaskan konsep rekursi: base case, recursive case, dan call stack

CPMK0609

Sub-CPMK 14.2

C3 – Menerapkan

Melakukan trace fungsi rekursif dan menganalisis alur eksekusinya

CPMK0609

Sub-CPMK 14.3

C3 – Menerapkan

Merancang struct/record dan menggunakannya dengan array serta fungsi

CPMK0609

Outline Pertemuan Hari Ini

	00-10	Review dan Hook	Recap fungsi + motivasi rekursi
	10-25	Konsep Rekursi	Base case, recursive case, analogi
	25-40	Trace Rekursi	Faktorial, Fibonacci, call stack
	40-55	Struct/Record	Tipe data bentukan, deklarasi, akses
	55-70	Workshop	Trace rekursi + desain struct
	70-80	Algorithm Detective	Bug: missing base case dan wrong field
	80-90	PBL#3 Milestone 2	Struct Mahasiswa + fungsi rekursif
	90-100	Exit Ticket dan Tugas	Assessment + homework



Pertanyaan Pemantik

*"Anda berdiri di antara dua cermin.
Bayangan di dalam bayangan di dalam bayangan... ∞"*

*"Buka folder, di dalamnya ada folder,
di dalamnya lagi ada folder... sampai mana?"*

*"Data mahasiswa punya nama, NIM, dan 5 nilai MK.
Bisakah disimpan di satu variabel?"*

Rekursi: Fungsi Memanggil Dirinya

Dua komponen wajib: base case + recursive case

Iteratif (Loop)

```
FUNGSI faktorial(n) : integer
  hasil ← 1
  UNTUK i ← 1 SAMPAI n
    hasil ← hasil * i
  AKHIR UNTUK
  RETURN hasil
AKHIR FUNGSI
```

Pendekatan: gunakan loop + accumulator

Rekursif (Self-Call)

```
FUNGSI faktorial(n) : integer
  JIKA n == 0 MAKA
    RETURN 1          ← base case
  SELAINNYA
    RETURN n * faktorial(n-1)
                          ← recursive case
  AKHIR FUNGSI
```

Pendekatan: pecah masalah jadi versi lebih kecil

2 Komponen WAJIB Rekursi

1. Base Case (Kondisi Berhenti)

Kondisi di mana fungsi BERHENTI memanggil dirinya. Tanpa base case = INFINITE RECURSION = crash!

Contoh: $n == 0 \rightarrow \text{return } 1$

2. Recursive Case (Langkah Rekursif)

Fungsi memanggil DIRINYA SENDIRI dengan parameter yang LEBIH KECIL. Harus menuju base case!

Contoh: $n * \text{faktorial}(n-1)$ — n berkurang 1 setiap pemanggilan

Trace: Rekursi Faktorial

$\text{faktorial}(4) = 4 \times 3 \times 2 \times 1 = 24$

```
FUNGSI faktorial(n) : integer
  JIKA n == 0 MAKA
    RETURN 1
  SELAINNYA
    RETURN n * faktorial(n-1)
  AKHIR FUNGSI
```

Panggilan (Going Down ↓)	Return (Going Up ↑)
faktorial(4)	$4 \times \text{faktorial}(3) = 4 \times 6 = 24$
faktorial(3)	$3 \times \text{faktorial}(2) = 3 \times 2 = 6$
faktorial(2)	$2 \times \text{faktorial}(1) = 2 \times 1 = 2$
faktorial(1)	$1 \times \text{faktorial}(0) = 1 \times 1 = 1$
faktorial(0)	RETURN 1 (base case!)
faktorial(4) = 24 — 5 pemanggilan, 5 return!	

Pola Rekursi: Call Stack (LIFO)

1. Setiap pemanggilan rekursif MASUK ke stack (push)
2. Base case tercapai → mulai RETURN (pop)
3. Return value dikirim ke pemanggil di atasnya
4. LIFO: Last In, First Out — faktorial(0) pertama return, faktorial(4) terakhir
5. Jumlah panggilan = $n+1$ (termasuk base case). Kompleksitas: $O(n)$ time, $O(n)$ space (stack)

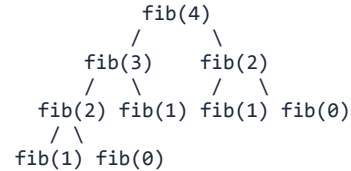
Rekursi: Fibonacci

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ — DUA pemanggilan rekursif!

```
FUNGSI fib(n) : integer
  JIKA n == 0 MAKA
    RETURN 0
  JIKA n == 1 MAKA
    RETURN 1
  RETURN fib(n-1) + fib(n-2)
AKHIR FUNGSI
```

Deret: 0, 1, 1, 2, 3, 5, 8, 13, ...

Call Tree: fib(4) = 3



Panggilan	n	Base?	Hitung	Return
fib(0)	0	YA	base case	0
fib(1)	1	YA	base case	1
fib(2)	2		$\text{fib}(1) + \text{fib}(0) = 1 + 0$	1
fib(3)	3		$\text{fib}(2) + \text{fib}(1) = 1 + 1$	2
fib(4)	4		$\text{fib}(3) + \text{fib}(2) = 2 + 1$	3

⚠ **Fibonacci rekursif = $O(2^n)$ SANGAT lambat! fib(50) = triliunan pemanggilan. Gunakan iteratif atau memoization!**

Struct/Record: Tipe Data Bentukkan

Gabungkan beberapa tipe data dalam satu wadah

✗ Tanpa Struct (Ribet!)

```
nama1 ← "Budi"  
nim1 ← "12345"  
ipk1 ← 3.75  
  
nama2 ← "Ani"  
nim2 ← "12346"  
ipk2 ← 3.90  
  
... (3 variabel per mahasiswa!)
```

✓ Dengan Struct (Rapi!)

```
STRUCT Mahasiswa  
  nama : string  
  nim  : string  
  ipk  : real  
AKHIR STRUCT  
  
mhs1.nama ← "Budi"  
mhs1.nim  ← "12345"  
mhs1.ipk  ← 3.75
```

Visualisasi: Struct = Kotak dengan Beberapa Laci

mhs1 : Mahasiswa

.nama	"Budi"
.nim	"12345"
.ipk	3.75

mhs2 : Mahasiswa

.nama	"Ani"
.nim	"12346"
.ipk	3.90

Array of Struct + Fungsi

Gabungkan struct, array, dan fungsi menjadi program modular!

```
STRUCT Mahasiswa
  nama : string
  nim  : string
  ipk  : real
AKHIR STRUCT

FUNGSI cariByNIM(daftar[], N, targetNIM) : integer
  UNTUK i ← 0 SAMPAI N-1
    JIKA daftar[i].nim == targetNIM MAKA
      RETURN i
    AKHIR JIKA
  AKHIR UNTUK
  RETURN -1
AKHIR FUNGSI

DEKLARASI mhs[3] : Mahasiswa
mhs[0].nama ← "Budi"   mhs[0].nim ← "12345"   mhs[0].ipk ← 3.75
mhs[1].nama ← "Ani"    mhs[1].nim ← "12346"   mhs[1].ipk ← 3.90
mhs[2].nama ← "Citra"  mhs[2].nim ← "12347"   mhs[2].ipk ← 3.50

idx ← cariByNIM(mhs, 3, "12346")
OUTPUT mhs[idx].nama           // Output: Ani
```

Workshop: Rekursi dan Struct

Kerjakan di kertas — 10 menit

Soal 1: Trace Rekursi

```
FUNGSI pangkat(base, exp) : integer
  JIKA exp == 0 MAKA
    RETURN 1
  RETURN base * pangkat(base, exp-1)
AKHIR FUNGSI
```

Pertanyaan:

- Trace pangkat(2, 4). Tulis setiap pemanggilan dan return value.
- Berapa total pemanggilan fungsi?
- Apa base case? Apa recursive case?
- Tulis versi ITERATIF dari fungsi ini.

Soal 2: Desain Struct

Rancang pseudocode untuk:

- Definisikan STRUCT MataKuliah:
 - kode : string (contoh: "IF1110")
 - nama : string (contoh: "Algoritma")
 - sks : integer (contoh: 2)
 - nilai : char (contoh: 'A')
- Buat array mk[3] : MataKuliah dan isi dengan 3 MK Informatika.
- Tulis FUNGSI hitungTotalSKS(mk[], N) : integer yang menghitung total SKS.
- Tulis FUNGSI hitungIPK(mk[], N) : real (A=4, B=3, C=2, D=1, E=0)

Jawaban Workshop

Soal 1: pangkat(2,4) = 16 dan Soal 2: Struct MataKuliah

Jawaban Soal 1: Trace

```
pangkat(2,4): return 2*pangkat(2,3)
pangkat(2,3): return 2*pangkat(2,2)
pangkat(2,2): return 2*pangkat(2,1)
pangkat(2,1): return 2*pangkat(2,0)
pangkat(2,0): return 1 (base!)
return 2*1 = 2
return 2*2 = 4
return 2*4 = 8
return 2*8 = 16 ✓
```

- b) 5 pemanggilan (exp+1)
- c) Base: exp==0. Recursive: base*pangkat(base,exp-1)

Jawaban Soal 2c: hitungTotalSKS

```
FUNGSI hitungTotalSKS(mk[], N) : integer
total ← 0
UNTUK i ← 0 SAMPAI N-1
    total ← total + mk[i].sks
AKHIR UNTUK
RETURN total
AKHIR FUNGSI
```

Akses field struct dengan titik: `mk[i].sks`

Jawaban Soal 2d: hitungIPK

```
FUNGSI nilaiKeAngka(huruf) : real
JIKA huruf == 'A' MAKA RETURN 4.0
JIKA huruf == 'B' MAKA RETURN 3.0
JIKA huruf == 'C' MAKA RETURN 2.0
JIKA huruf == 'D' MAKA RETURN 1.0
RETURN 0.0
```

```
FUNGSI hitungIPK(mk[], N) : real
totalBobot ← 0
totalSKS ← 0
UNTUK i ← 0 SAMPAI N-1
    totalBobot ← totalBobot + nilaiKeAngka(mk[i].nilai) * mk[i].sks
    totalSKS ← totalSKS + mk[i].sks
RETURN totalBobot / totalSKS
```



Algorithm Detective

Temukan bug pada rekursi dan struct berikut!

Kasus 1: Missing Base Case

```
FUNGSI countdown(n)
  OUTPUT n
  countdown(n - 1)
AKHIR FUNGSI
```

Bug: TIDAK ADA BASE CASE!
countdown(5) → countdown(4) →
countdown(3) → ... → countdown(-999)
→ STACK OVERFLOW!

Fungsi tidak pernah berhenti.
Setiap panggilan menambah stack.
Stack penuh = CRASH!

Fix: tambahkan base case!
JIKA $n < 0$ MAKA
 RETURN
AKHIR JIKA

Kasus 2: Wrong Field Access

```
STRUCT Mahasiswa
  nama : string
  nim   : string
  ipk   : real
AKHIR STRUCT

mhs.name ← "Budi"
OUTPUT mhs.NIM
```

Bug 1: mhs.name bukan mhs.nama
Field harus PERSIS sesuai definisi struct.
"name" tidak ada — yang ada "nama"!

Bug 2: mhs.NIM bukan mhs.nim
Field bersifat CASE SENSITIVE.
"NIM" ≠ "nim"!

Fix: selalu cek definisi struct.
Gunakan nama field yang konsisten.



Aturan Emas Rekursi dan Struct

Hindari bug yang paling umum

1

Rekursi HARUS punya base case

Tanpa base case = infinite recursion = stack overflow = crash. Selalu tulis base case PERTAMA.

JIKA kondisi MAKA RETURN

2

Parameter harus menuju base case

Setiap panggilan rekursif: parameter harus LEBIH KECIL ($n-1$, $n/2$). Jika tidak, tidak akan berhenti.

`pangkat(base, exp-1)`

3

Struct field CASE SENSITIVE

nama $\neq$ Nama $\neq$ NAME. Gunakan field persis seperti definisi struct. Konsisten!

Cek definisi struct!

4

Akses field dengan titik (.)

mhs.nama, mhs.ipk, arr[i].nim. Array of struct: indeks dulu, lalu titik.

`arr[i].field`

PBL#3: Student Record System — Milestone 2

Struct Mahasiswa + fungsi rekursif

Milestone 2: Struct + Rekursi (P14)

✓ Milestone 1 selesai (desain fungsi modular)

Tambahkan ke Student Record System:

- Definisikan STRUCT Mahasiswa (nama, NIM, array nilai[], IPK)
- Buat array mhs[N] : Mahasiswa untuk menyimpan data
- Implementasi fungsi hitungIPK menggunakan struct
- Minimal 1 fungsi REKURSIF (contoh: binary search rekursif, atau hitungTotal rekursif, atau cetak data rekursif)

Timeline PBL#3

P13

Desain fungsi modular: hitungIPK, cariMhs, sortByIPK

Milestone 1 ✓

P14

Struct Mahasiswa + fungsi rekursif

Milestone 2 📌

P15

Review dan integrasi → SUBMIT

Deadline!

Ringkasan Pertemuan 14

Rekursi = fungsi memanggil dirinya

Dua komponen wajib: base case (berhenti) + recursive case (panggil diri, parameter lebih kecil).

Trace rekursif = Call Stack (LIFO)

Push saat panggil, pop saat return. faktorial(4): 5 panggilan, 5 return.



Fibonacci rekursif = $O(2^n)$

Sangat lambat! Gunakan iteratif atau memoization untuk efisiensi.

Struct = tipe data bentukan

Gabungkan nama, NIM, IPK dalam 1 wadah. Akses dengan titik: mhs.nama

PBL#3 Milestone 2

Struct Mahasiswa + fungsi rekursif. Deadline submit di P15!

Exit Ticket — Pertemuan 14

10 soal • 15 menit • kerjakan di kertas

1 Apa 2 komponen wajib dalam fungsi rekursif? Jelaskan!

2 Trace faktorial(5). Tulis setiap panggilan dan return.

3 Apa yang terjadi jika rekursi TIDAK punya base case?

4 Trace fib(5). Berapa hasilnya? Berapa total pemanggilan?

5 Kenapa Fibonacci rekursif lambat? Berapa kompleksitasnya?

6 Definisikan STRUCT Buku (judul, penulis, tahun, harga).

7 Buat array bk[3]:Buku. Akses judul buku ke-2. Pseudocode!

8 Tulis fungsi rekursif jumlah(n):integer = $1+2+\dots+n$.

9 Apa perbedaan rekursi dan iterasi? Kapan pakai rekursi?

10 FUNGSI cariMax rekursif untuk array. Tulis pseudocode!

Tugas Mingguan — Rekursi dan Struct

Deadline: sebelum Pertemuan 15 • Kerjakan di kertas

Tugas A: Individual — Trace dan Desain

Soal 1: Trace pangkat(3, 4) rekursif. Tulis call stack lengkap!

Soal 2: Tulis fungsi rekursif:

- (a) jumlahDigit(n) : integer — jumlah digit angka (contoh: 1234 → 10)
 - (b) balikString(s) : string — balik string (contoh: "ITERA" → "ARETI")
- Tentukan base case dan recursive case!

Soal 3: Definisikan STRUCT Karyawan (nama, jabatan, gaji, lamaKerja).

Tulis FUNGSI cariGajiMax(kar[], N) : real
dan FUNGSI hitungRataGaji(kar[], N) : real.

Tugas B: Kelompok — PBL#3 Milestone 2

Implementasi struct Mahasiswa + array of struct + fungsi hitungIPK + minimal 1 fungsi rekursif.

Siapkan pseudocode lengkap dan trace table. Persiapkan untuk SUBMIT di P15!

Referensi Pertemuan 14

Buku

1. Deitel — C++ How to Program, Bab 6.16: Recursion
2. Gaddis — Starting Out with C++, Bab 14: Recursion
3. Cormen (CLRS) — Introduction to Algorithms, Bab 4: Divide dan Conquer

Online

1. visualgo.net/recursion — Visualisasi rekursi interaktif
2. cs50.harvard.edu — Lecture 5: Data Structures
3. [geeksforgeeks.org/recursion](https://www.geeksforgeeks.org/recursion) — Recursion Tutorial



Terima Kasih

IF25-11001 Algoritma Pemrograman

Pertemuan 14: Rekursi dan Struct

Program Studi Teknik Informatika
Institut Teknologi Sumatera
2026